

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA EN INFORMÁTICA Y

SISTEMAS



“Proyecto Mini Shell en C++”

ASIGNATURA:

Sistemas Operativos

DOCENTE:

Hugo Manuel Barraza Vizcarraq

INTEGRANTES:

Edu Rubinho Puma Ccama	2023-119053
Vladimir Roger Ticona Mamani	2023-119063
Obando Huallpa Jorge Enrique	2017-10045

TACNA-2025

1. Objetivo y alcance.....	3
1.1. Objetivo general.....	3
1.2. Objetivos específicos.....	3
2. Arquitectura y diseño.....	3
2.1. Arquitectura General.....	3
2.2. Manejo de I/O.....	4
2.3. Diagrama de Procesos y hilos.....	5
3. Detalles de implementación.....	6
3.1. Funciones Principales:.....	6
3.2. Manejo de Procesos:.....	6
3.3. Redirección de Entrada y Salida:.....	6
3.4. Concurrencia:.....	6
3.5. Gestión de Memoria:.....	6
4. Concurrencia y sincronización.....	6
4.1. Paralelización de Comandos:.....	6
4.2. Sincronización.....	7
4.3. Evitar Condiciones de Carrera:.....	7
5. Gestión de memoria.....	7
5.1. Estrategia de Gestión de Memoria:.....	7
6. Pruebas y resultados.....	7
7. Conclusiones.....	8
8. Anexos.....	8

1. Objetivo y alcance

1.1. Objetivo general

Desarrollar un intérprete de comandos (mini-shell) en C++ para sistemas Linux, implementando el manejo de procesos, hilos, concurrencia y gestión de memoria, como se establece en la Unidad I del curso.

1.2. Objetivos específicos

- Crear un shell que ejecute comandos mediante `fork()` y `exec*()`, manejando la sincronización con `wait()` y `waitpid()`.
- Implementar redirección de salida estándar utilizando `dup2`, `pipe` y `close`.
- Utilizar mecanismos de concurrencia como hilos (`pthread`) para la ejecución en paralelo de comandos.
- Manejar errores de ejecución y redirección de salida, mostrando mensajes claros.
- Implementar comandos internos como `cd`, `pwd`, `help`, `history` y alias.
- Integrar estadísticas de uso de memoria (manejadas a través de `/proc/self/status`).
- Garantizar la seguridad de la memoria con un manejo adecuado de `heap`.

2. Arquitectura y diseño

La arquitectura del mini-shell desarrollado sigue un modelo basado en procesos e hilos. El diseño se enfoca en la ejecución de comandos de forma concurrente, el manejo de entrada/salida (I/O) y la correcta gestión de la memoria y procesos hijos. A continuación se explica cómo funciona y se organiza este diseño:

2.1. Arquitectura General

■ Proceso Principal (Shell)

- Es el proceso principal que ejecuta el mini-shell.
- Recibe la entrada del usuario, la procesa (tokenización, interpretación de comandos), y luego decide si ejecutar comandos internos o externos.
- Si es un comando interno (como `cd`, `pwd`, etc.), lo ejecuta en el mismo proceso.
- Si es un comando externo, se crea un proceso hijo para ejecutar el comando.

■ Proceso Hijos

- Se crean con `fork()` cuando el comando es externo. El proceso hijo ejecuta el comando usando `execv()` y se comunica con el proceso padre mediante el código de retorno.
- El proceso padre usa `waitpid()` o `wait()` para esperar a que el proceso hijo

■ Hilos (Concurrencia)

- Para comandos que se ejecutan en paralelo (como la extensión de ejecución paralela), se crean hilos usando `pthread_create()`.
-
- Los hilos permiten ejecutar múltiples comandos al mismo tiempo, lo que mejora la eficiencia y reduce el tiempo de espera cuando se manejan múltiples tareas simultáneamente.

2.2. Manejo de I/O

El manejo de I/O en el mini-shell se realiza de la siguiente manera:

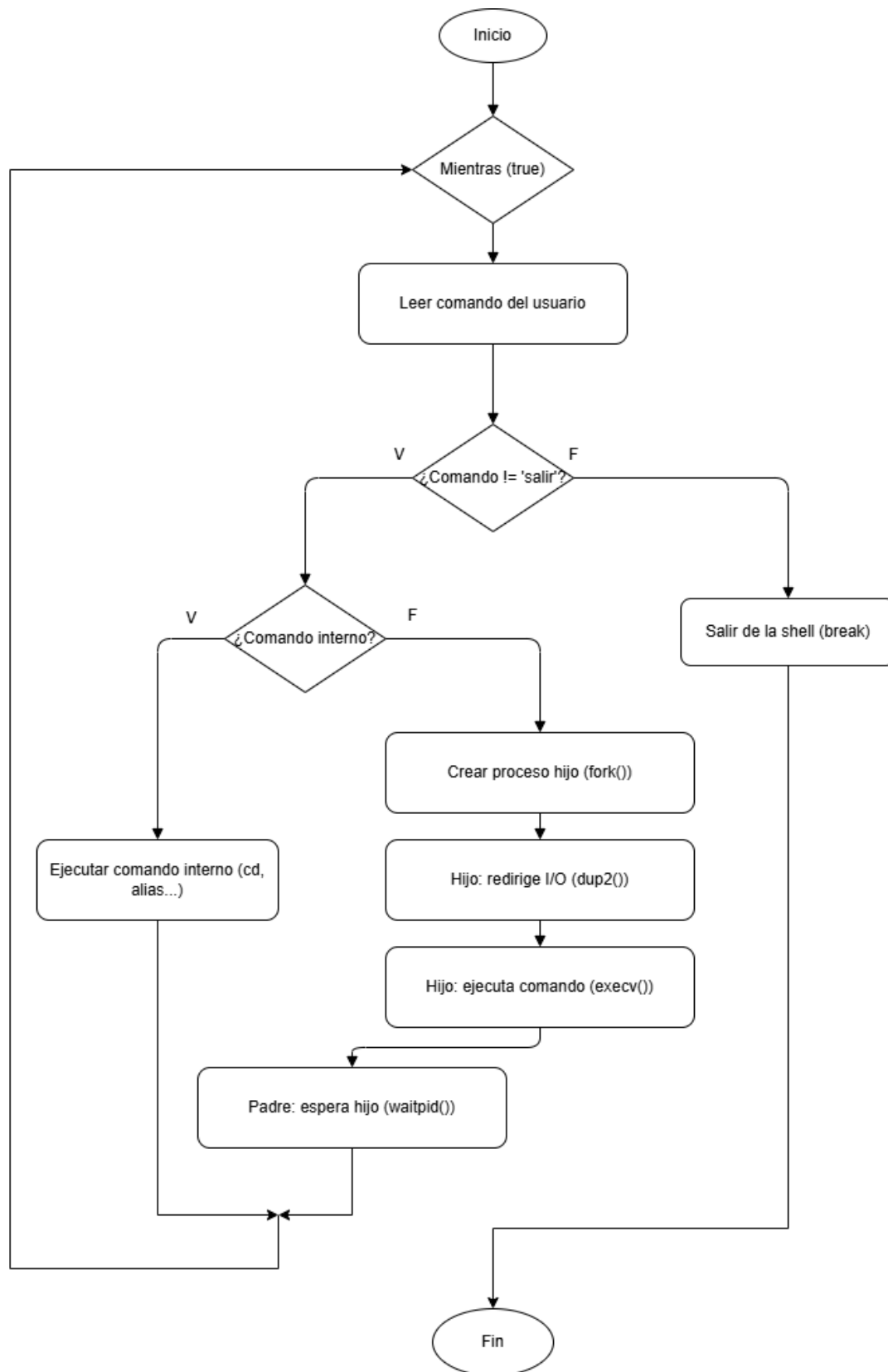
■ Entrada

- La entrada es recibida mediante el uso de `getline()`, que captura el comando completo ingresado por el usuario.
- La entrada es procesada (tokenizada) usando la función `tokenizar()`, que divide la línea en los diferentes tokens (comando y argumentos).

■ Salida

- Si el comando es ejecutado de forma normal, su salida se imprime en la consola (`stdout`).
- Si se utiliza redirección de salida (`>`), la salida se redirige a un archivo utilizando `dup2()`.
- Si se utiliza redirección de entrada (`<`), la entrada se redirige desde un archivo usando también `dup2()`.

2.3. Diagrama de Procesos y hilos



3. Detalles de implementación

3.1. Funciones Principales:

- `mostrar_prompt()`: Muestra el prompt personalizado con el directorio actual.
- `tokenizar()`: Separa la línea de entrada en tokens utilizando espacios y tabulaciones.
- `builtin_cd()`: Permite cambiar el directorio de trabajo.
- `builtin_pwd()`: Muestra el directorio de trabajo actual.
- `builtin_help()`: Muestra los comandos disponibles y su descripción.
- `builtin_history()`: Muestra el historial de comandos ejecutados.
- `builtin_meminfo()`: Muestra estadísticas de memoria del proceso.
- `builtin_alias()` y `builtin_unalias()`: Gestionan alias de comandos.

3.2. Manejo de Procesos:

Los comandos externos se ejecutan en procesos hijos creados con `fork()`. En el proceso hijo, se utiliza `execv()` para ejecutar el comando y `waitpid()` en el proceso padre para esperar la finalización del comando.

3.3. Redirección de Entrada y Salida:

Se implementa la redirección de salida utilizando `dup2()` y `open()` para dirigir la salida de un comando a un archivo en lugar de la consola.

3.4. Concurrencia:

Se implementa un comando `parallel` que permite ejecutar múltiples comandos en paralelo utilizando hilos (`pthread_create()`). Los hilos se sincronizan usando `mutex` y variables de condición.

3.5. Gestión de Memoria:

El comando `meminfo` extrae información de uso de memoria de `/proc/self/status` y muestra estadísticas como el uso del `heap`, el tamaño de la memoria virtual y física, etc.

4. Concurrencia y sincronización

4.1. Paralelización de Comandos:

Se implementa un comando `parallel` que permite ejecutar varios comandos en paralelo. Esto se logra mediante la creación de hilos con `pthread_create()`, donde cada hilo ejecuta un comando y se sincroniza usando `mutexes` para garantizar que no se produzcan condiciones de carrera.

4.2. Sincronización

El acceso a recursos compartidos (como la lista de historial o los alias) se maneja con variables de condición y mutex para evitar interferencias entre los hilos.

4.3. Evitar Condiciones de Carrera:

Se asegura que las operaciones en recursos compartidos sean atómicas, utilizando sincronización adecuada, como bloqueos de mutex, para evitar la corrupción de datos durante la ejecución de comandos en paralelo.

5. Gestión de memoria

5.1. Estrategia de Gestión de Memoria:

Se utiliza malloc() y free() para gestionar la memoria dinámica. Las estructuras de datos como el historial de comandos y los alias se gestionan dinámicamente.

El comando meminfo consulta el archivo /proc/self/status para obtener estadísticas sobre el uso de memoria del proceso y mostrar información sobre el consumo del heap y otros recursos del sistema.

6. Pruebas y resultados

Caso de Prueba	Descripción	Resultado Esperado	Resultado Obtenido
Comando "cd" y "pwd"	Verificar el cambio de directorio y la correcta visualización del directorio actual.	El comando cd debe cambiar el directorio correctamente, y pwd debe mostrar el directorio actual después del cambio.	El comando cd cambia el directorio correctamente, y pwd muestra el directorio esperado.
Redirección de salida con ">"	Verificar que la salida de los comandos se redirige correctamente a un archivo.	Al usar comando > archivo, la salida del comando debe ser redirigida al archivo especificado sin	La redirección de salida funciona correctamente, el archivo se crea y contiene la salida esperada.

		mostrarla en consola.	
Alias	Crear y eliminar alias, asegurando que el sistema maneje correctamente los comandos personalizados.	Los alias deben ser creados con alias nombre='comando' y eliminados con unalias nombre. El alias debe ejecutarse correctamente.	Los alias son creados y eliminados correctamente. El comando alias se ejecuta sin problemas.
Comando "parallel"	Ejecutar múltiples comandos en paralelo y verificar que no haya condiciones de carrera.	Los comandos ejecutados en paralelo con parallel deben completarse sin interferencias, y los resultados deben ser correctos.	Los comandos paralelos se ejecutan correctamente y de forma independiente, sin condiciones de carrera.
Meminfo	Verificar que las estadísticas de memoria se muestran correctamente.	El comando meminfo debe mostrar las estadísticas correctas de memoria del proceso, obtenidas de /proc/self/status.	Los comandos paralelos se ejecutan correctamente y de forma independiente, sin condiciones de carrera.

7. Conclusiones

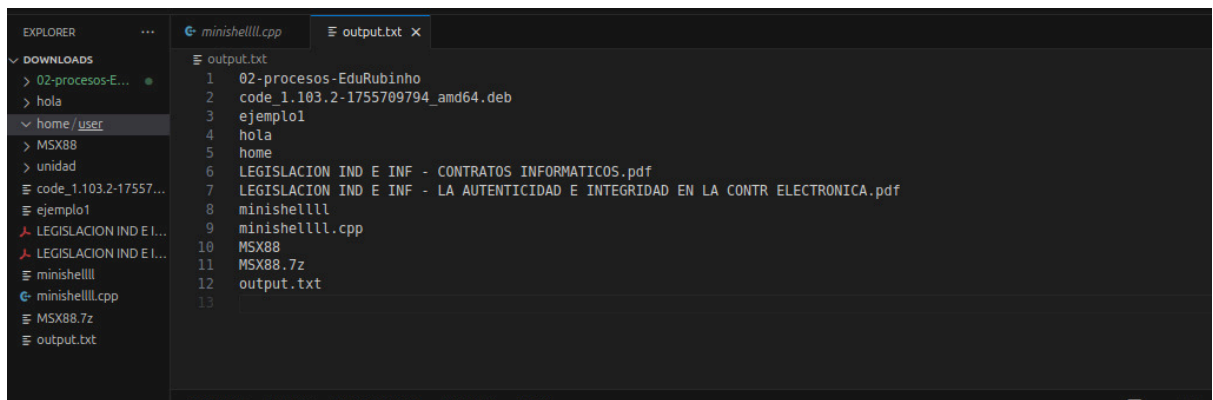
El mini-shell cumple con los objetivos establecidos, implementando correctamente las funcionalidades básicas y avanzadas, incluyendo redirección de salida, concurrencia, y gestión de memoria. Se logró una correcta sincronización de hilos y una gestión eficiente de recursos. Se podría extender la funcionalidad con más comandos internos, como kill para terminar procesos, o mejoras en la gestión de memoria, implementando contadores de referencias o utilizando técnicas más avanzadas como mmap para gestionar grandes cantidades de memoria de manera eficiente.

8. Anexos

```
ubuntu_edu@ubuntu-edu:~/Downloads$ ./minishelllll
=====
Bienvenido a Mini-Shell
Escribe 'help' para ver la ayuda
Escribe 'salir' para terminar
=====

mini-shell [/home/ubuntu_edu/Downloads]> pwd
/home/ubuntu_edu/Downloads
mini-shell [/home/ubuntu_edu/Downloads]> ls > output.txt
mini-shell [/home/ubuntu_edu/Downloads]> mkdir unidad
mini-shell [/home/ubuntu_edu/Downloads]> alias m='ls'
Alias creado: m -> ls
mini-shell [/home/ubuntu_edu/Downloads]> m
02-procesos-EduRubinho          hola          'LEGISLACION IND E INF - LA AUTENTICIDAD E INTEGRIDAD
EN LA CONTR ELECTRONICA.pdf'    MSX88          unidad
code_1.103.2-1755709794_amd64.deb home          minishelllll
                                MSX88.7z
ejemplo1                        'LEGISLACION IND E INF - CONTRATOS INFORMATICOS.pdf' minishelllll.cpp
                                output.txt
mini-shell [/home/ubuntu_edu/Downloads]> echo "hola mundo" > salida.txt
mini-shell [/home/ubuntu_edu/Downloads]> cat salida.txt
"hola mundo"
mini-shell [/home/ubuntu_edu/Downloads]> █
```

Anexo 01: Se muestra pruebas de la “Mini-shell”



```
EXPLORER  ...  minishelllll.cpp  output.txt x
└─ DOWNLOADS
  > 02-procesos-EduRubinho
  > hola
  └─ home / user
    > MSX88
    > unidad
    └─ code_1.103.2-17557...
      └─ ejemplo1
        └─ LEGISLACION IND E I...
          └─ LEGISLACION IND E I...
            └─ minishelllll
              └─ minishelllll.cpp
                └─ MSX88.7z
                  └─ output.txt
```

Anexo 02: Se muestra resultados de la “Mini-shell”